

20250711 日报

今日工作内容

1. 和岳哥再次对齐后，对程序再次进行了修改，实现后：以七彩石名称为文件夹名；以七彩石配置的键为本地文件名；

主要修改点：

- 插件中不做解析，解析操作在主逻辑中，插件只从本地文件返回[]byte;
- 以 trpc_go.yaml 中的七彩石 name 为文件夹名，以配置项的 key 为文件名（可以没有后缀）;

主要逻辑代码：

```

var channelSize = 1024 // 监听 channel 大小
const (
    pluginType = "config"
    pluginName = "rainbow"
)

func init() {
    plugin.Register(pluginName, &Plugin{})
}

// Config-rainbow 配置
type Config struct {
    StandbyRainbow bool          `yaml:"standbyrainbow"`
    Providers       []*rainbow.Config `yaml:"providers"`
}

// Plugin 插件
type Plugin struct {
}

// Type 插件的类型
func (p *Plugin) Type() string {
    return pluginType
}

// Setup 装载插件
func (p *Plugin) Setup(name string, dec plugin.Decoder) error {
    if dec == nil {
        return fmt.Errorf("standbyrainbow config decoder empty")
    }
    cfg := &Config{}
    if err := dec.Decode(cfg); err != nil {
        return err
    }
    // 根据开关配置动态路由
    if !cfg.StandbyRainbow {
        return p.setupPrimaryRainbow(name, dec) // 调用原始七彩石
    }
    // 备用七彩石
    log.Info("Using standby rainbow implementation")
    for i, provider := range cfg.Providers { // 处理所有七彩石配置
        if provider != nil {

```

```

type StandbyKV struct {
    name      string
    configs   map[string]string
    watcher   *fsnotify.Watcher
    listeners map[string]chan config.Response
    mu        sync.Mutex // watcher 需要上锁
}

// Name 返回七彩石插件名称
func (k *StandbyKV) Name() string {
    return k.name
}

// Get 读取指定七彩石配置
func (k *StandbyKV) Get(ctx context.Context, key string,
    opts ...config.Option) (config.Response, error) {
    val, exists := k.configs[key]
    if !exists {
        return nil, config.ErrConfigNotExist
    }
    return &standbyResponse{
        value: val,
        event: config.EventTypePut, // set or update event
    }, nil
}

func (k *StandbyKV) Put(_ context.Context, _ string, _ string,
    _ ...config.Option) error {
    return fmt.Errorf("standby mode does not support put")
}

func (k *StandbyKV) Del(_ context.Context, _ string,
    _ ...config.Option) error {
    return fmt.Errorf("standby mode does not support del")
}

func (k *StandbyKV) initWatcher(key string) error {
    k.mu.Lock()
    defer k.mu.Unlock()
    if k.watcher != nil {
        return nil
    }
    watcher, err := fsnotify.NewWatcher()

```

```

func setupKV(directoryName string, i int) error {
    // 存储文件名
    configMap := make(map[string]string)
    err := filepath.Walk(directoryName, func(path string, info
os.FileInfo, err error) error {
        if err != nil {
            return fmt.Errorf("access path %s failed: %w", path,
err)
        }
        if info.IsDir() || info.Size() == 0 {
            return nil // 跳过目录和空文件
        }
        fileName := filepath.Base(path)
        fmt.Println(fileName)
        data, err := os.ReadFile(path)
        stringdata := string(data)
        if err != nil {
            return fmt.Errorf("read file %s failed: %w", path, err)
        }
        configMap[fileName] = stringdata
        kv := &StandbyKV{
            name:    directoryName,
            configs: configMap,
        }
        config.Register(kv)
        if i == 0 {
            config.SetGlobalKV(kv)
        }
        log.Infof("Loaded %d config files from %s",
len(configMap), directoryName)
        return nil
    })
    if err != nil {
        return fmt.Errorf("traverse directory failed: %w", err)
    }
    return nil
}

```

测试结果：

```
test > standByrainbow_kv > {} rainbow_kv.json > {} hobbies
1 {
2   "address": {
3     "city": "New York",
4     "street": "123 Main St"
5   },
6   "age": 30,
7   "hobbies": [
8     "reading",
9     "coding",
10    "travelling"
11  ],
12  "isStudent": false,
13  "name": "John Doe"
14 }
15

test > standByrainbow_kv > {} rainbow_copy_kv > {} personinfo > # age
1 personinfo:
2   name: drewjjli
3   age: 27
4   hobbies:
5     ball: basketball
6     music: rap
7

2025-07-11 19:50:10.354 INFO trpc-naming:polaris@0.5.24/naming.go:231 naming-polaris starts to set polaris-go logs level: false
plugin selector polaris setup succeed, time elapsed: 12.723738ms
2025-07-11 19:50:10.367 DEBUG maxprocs: maxprocs.go:48 maxprocs: Updating GOMAXPROCS-16: determined from CPU quota
2025-07-11 19:50:10.368 INFO trnet/tcp-transport-go:100 service: hello.Greeter is using trnet tcp transport, current number of pollers: 1
map[address:map[city:New York street:123 Main St] age:30 hobbies:[reading coding travelling] isStudent:false name:John Doe]
K[drewjjli 27] [basketball rap]
2025-07-11 19:50:10.369 INFO server/service-go:207 process: 1024987, trpc service: hello.Greeter launch success, tcp: 21.6.68.166:8800, serving ...
2025-07-11 19:50:10.408 INFO trnet@0.1.0/trpservice.go:88 trnet tcp service started, current number of pollers: 1, use trnet.SetNumPollers to change it
```

今日所遇问题：

- 因为我之前的代码逻辑是在插件 Setup()阶段就解析为具体的文件格式，如 json、yaml 等，这样的话就需要去判断本地文件的格式。但其实不然，真正的解析是在主逻辑代码中进行的，所以代码需要改正。

当前需求进度：

- 基本已经完成 KV 类型的各种文件类型（json、yaml 等）七彩石配置的支持；
- 可以实时监听指定文件；

明日待办：

1. 在当前进度的基础上，在七彩石正常启动时需要实时更新配置文件到本地；
2. 在当前进度的基础上，完成 file、table、group 类型的逻辑；